

DAST

Gabriele Daga

February 10, 2025

Abstract

In today's interconnected world, smart sensors and advanced communication technologies are integral to various sectors, capturing raw data from the physical environment. However, these sensors are susceptible to malfunctions, failures, and malicious attacks, leading to erroneous and conflicting readings. Such compromised data can degrade the performance of systems and corrupt trained Machine Learning (ML) models. This underscores the necessity for robust sensor anomaly detection mechanisms to ensure data integrity.

While traditional Industrial Control Systems (ICS) and Industrial Internet of Things (IIoT) setups have been the primary focus of anomaly detection research, this project explores a different avenue by utilizing satellite sensor data. Unlike terrestrial sensors in IIoT environments, satellite sensors operate in space, providing global coverage and capturing data across vast and remote areas.

The primary objective of this project is to develop an anomaly detection system specifically targeted for satellites. By analyzing spatial and temporal correlations in the data, the system aims to detect deviations indicative of potential intrusions or data corruption.

Through this work, I proposed the adoption of a novel method to ensure the resilience of satellite data against data poisoning attacks by effectively detecting them.

Contents

1	Introduction	2
1.1	Securing Satellite Data	2
1.1.1	Related work on satellite data	2
1.1.2	Related work in IoT domain	2
1.1.3	Differences with satellite data	3
1.1.4	Potential Attacks on Earth Observation Devices	3
2	The Dataset	4
2.1	Challenges in Acquiring Datasets for Anomaly Detection	4
2.2	Overview of NASA Data Repository	4
2.3	Selected Datasets: Collection, Representation, and Structure	5
2.3.1	NCEP/CPC dataset exploration	7
2.3.2	Potential attacks	8
3	The Anomaly Detector	10
3.1	Proposed Solution: Unsupervised Training of STGAE	10
3.1.1	Graph Neural Networks	10
3.1.2	Graph Auto Encoders	11
3.1.3	Spatio-Temporal Graph Auto Encoders	12
3.1.4	The proposed solution	12
3.1.5	Evaluation Metrics	14
3.1.6	Results and Analysis	15
4	Conclusion	16
4.1	Summary of Findings	16
4.2	Improvements and Future Developments	16

Chapter 1

Introduction

1.1 Securing Satellite Data

Satellite-based data is critical infrastructure, underpinning everything from weather forecasting to national security. However, this reliance creates a significant vulnerability, exacerbated by the rapidly evolving cyber threat landscape and the increasing sophistication of attacks targeting space-based assets. This project isn't just about detecting data tampering; it's driven by a deep concern about the potential consequences of compromised satellite data. By analyzing anomalies in satellite data patterns, I aim to develop and validate an intrusion detection system (IDS) that can safeguard the integrity of the data used for critical forecasting and analysis. The thought of inaccurate weather predictions or compromised national security due to manipulated data is what fuels this work.

1.1.1 Related work on satellite data

Satellite system anomalies can arise from various sources, including environmental factors, hardware failures or malicious activities. Detecting these anomalies promptly is important to prevent potential failures and ensure the reliability of the data collected during satellite missions.

By analyzing patterns and identifying deviations from established norms, these techniques can flag potential problems before they escalate.

Traditional monitoring approaches, often relying on static thresholds, fail to capture subtle or dynamic anomalies due to the vast amount of telemetry data generated and the complex, evolving nature of satellite operations. Traditional monitoring approaches often fail to identify subtle or unforeseen anomalies due to the vast amount of telemetry data generated and the dynamic nature of satellite operations [6]. However, a recent study [9] introduced a sparse feature-based anomaly detection method that effectively identifies hybrid anomalies in satellite telemetry data, enhancing the precision of monitoring systems.

1.1.2 Related work in IoT domain

In some recent works carried out on anomaly detection in IIoT scenarios (such as by [1], [11] and [10]), the authors have exploited mainly two concepts:

1. **Spatial correlation:** Clients that are physically closer tend to share similar properties or behaviors.
2. **Temporal correlation:** Describes the behavior of data sampled over time by each client, where values tend to follow a continuous trend without sudden changes or imbalances.

In some recent studies [4], researchers have modeled sensors networks as graphs, in order to better capture spatial and temporal correlation between sensors.

In such a structure, $G(V, E)$, each sensor or client is represented as a node $i \in V$. E is the set of edges, and $e_{i,j} \in E$ represents a connection between vertices i and $j \in V$. If two nodes i and j are connected, $e_{i,j} = 1$, otherwise $e_{i,j} = 0$. In general, these connections can be represented by an adjacency matrix. This natural representation of spatial correlation allows for efficient detection of anomalies that deviate from the behavior of neighboring nodes. Moreover, by integrating temporal features into the node attributes or by employing dynamic graphs that evolve over time, one can capture the temporal correlations inherent in the data.

Consequently, graph-based models provide a flexible framework to model both spatial and temporal dependencies, enhancing the capability of anomaly detection systems in complex IoT environments.

1.1.3 Differences with satellite data

Satellite data and Internet of Things (IoT) sensor have quite noticeable differences:

- While the frequency of data collection by satellites depends on their orbit and specific mission parameters, often resulting in periodic data acquisition with intervals that can range from minutes to days, IoT sensors can provide continuous or high-frequency data streams.
- Another key difference lies in the coverage area: satellites cover broad areas, but the spatial resolution may be coarser (depending on the sensor capabilities and altitude), whereas IoT sensors can provide highly granular data specific to their immediate environment, allowing for detailed analysis.

Although these are two very different domains, the scope of my work was to adopt the very same approach for satellite data, namely exploiting the same concept of spatial and temporal correlation.

1.1.4 Potential Attacks on Earth Observation Devices

Despite advancements in securing satellite communications, these systems remain vulnerable to various cyber threats. Attackers can exploit weaknesses in satellite networks through methods such as signal jamming and spoofing, where false signals are generated or legitimate communications are disrupted. More sophisticated attacks may involve unauthorized access to satellite systems, potentially allowing adversaries to take control of the satellites themselves.

Additionally, many satellite communication providers do not offer encryption below the application layer by default, making their services susceptible to eavesdropping and data interception [8]. The increasing reliance on satellite communications in critical infrastructure underscores the importance of robust cybersecurity measures to protect against evolving threats, such as:

- **Data Tampering:** Attackers may intercept and alter the data transmitted from satellite sensors to ground stations. By breaching unprotected data links, they can manipulate the information, leading to incorrect analyses and potentially harmful decisions.
- **Sensor Spoofing:** This involves feeding false signals to satellite sensors, causing them to record inaccurate data. For instance, adversaries can target sensor readings in specific bands, such as the infrared, to affect the received datasets and the computed positions.
- **Jamming and Denial of Service:** By overwhelming satellite communication channels with noise or malicious signals, attackers can disrupt the availability of data, leading to gaps in critical information and hindering timely decision-making.
- **Adversarial Attacks on Machine Learning Models:** Many modern satellites use machine learning for tasks like cloud detection. Adversaries can exploit vulnerabilities in these models by introducing subtle perturbations to the input data, causing misclassifications that lead to incorrect data interpretation.

Chapter 2

The Dataset

2.1 Challenges in Acquiring Datasets for Anomaly Detection

In the field of Anomaly Detection, it is hard to find datasets for supervised learning, and even to build ones. Although there are some (like the MSL dataset), they are focused more on anomalies in its strict meaning (a.k.a. sensors malfunctions), but they are not thought to be used to detect intrusions and/or tampering. Due to the lack of labeled data in this field, an unsupervised approach is preferable. Numerous sensor measurement datasets have been employed to learn normal patterns and detect outliers [3]. This method enables flexible anomaly detection workflows, even when anomaly detection was not the primary objective.

A novel method, previously applied to IoT networks, trains a neural network (typically an auto-encoder) on normal data so that anomalous inputs yield high reconstruction errors. Recently, this approach has been integrated with Graph Neural Networks, allowing detection in scenarios where data collection was not originally intended for anomaly analysis. In these workflows, each sensor is treated as a node, capturing both temporal behavior and spatial correlations. This method will be employed in this project and discussed further in the next chapter.

For what concerns satellite data, NASA provides a comprehensive list of earth and space observation datasets via its web portal, which may satisfy these requirements.

2.2 Overview of NASA Data Repository

NASA's data repository allows users to search for specific features in the datasets, namely:

- Spatial resolution: how much surface a single observation covers
- Temporal resolution: frequency of measurements
- Coverage time: the period of time covered by the dataset
- Measurement type: the type of observed data (e.g. air temperature, precipitation measurement, etc)
- Mission name: the mission of the dataset
- Processing level: Datasets are categorized into different processing levels, each representing a stage in data refinement:
 - Level 0: Raw, unprocessed instrument data.
 - Level 1: Reconstructed, unprocessed instrument data at full resolution, time-referenced, and annotated with ancillary information.
 - Level 2: Derived geophysical variables at the same resolution and location as Level 1 source data.
 - Level 3: Variables mapped on uniform space-time grids, usually with some completeness and consistency.
 - Level 4: Model output or results from analyses of lower-level data.

To select the most suitable dataset for my project, I drew some priorities, namely:

1. The ratio $\frac{\text{measurement_period}}{\text{measurement_frequency}}$ should be as big as possible, in order to be able to capture as more data sequences as possible.
2. The spatial resolution should be small enough to be able to cover a denser area, and thereby success in spotting anomalous data distributions.
3. The stage in data refinement should not be higher than 3. In fact, while in level 0 datasets the data is at the very first stage and is not even preprocessed and cleaned, levels 4 and beyond are the result of some re-analysis processes, that could mask and hide anomalies and tampering attempts carried at previous steps. The anomaly detection process should be then carried out in an intermediate phase, when data has been properly cleaned, but also has not been altered too much from its initial state.

2.3 Selected Datasets: Collection, Representation, and Structure

This first selection led me to narrowing down my research to mainly two datasets:

1. GPM (IMERG) level 1B : This dataset, produced by the Global Precipitation Measurement (GPM) mission—a collaboration between NASA, JAXA, and international partners—advances our understanding of Earth’s precipitation. It employs a Dual-Frequency Precipitation Radar (DPR) that includes:
 - Ku-band Precipitation Radar (KuPR): Operating at 13.6 GHz, it provides high-resolution measurements of precipitation intensity and structure.
 - Ka-band Precipitation Radar (KaPR): Operating at 35.5 GHz, it enhances sensitivity to small raindrops and light precipitation.
 - The Level-1B products are organized by swaths and timestamps, containing:
 - KuPR Data: Received power profiles and geometric information (e.g., observation positions).
 - KaPR Data: Similar power profiles and geometric details for comprehensive precipitation analysis.

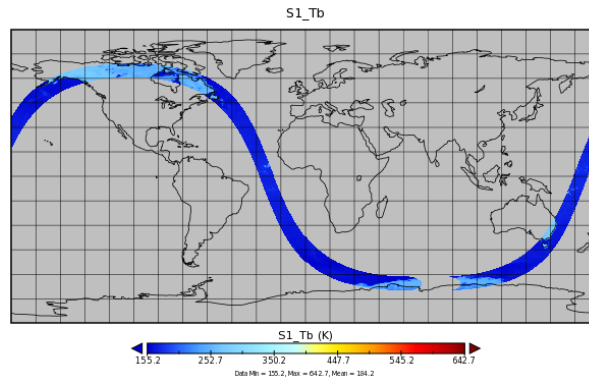


Figure 2.1: Representation of the GPM (IMERG) level 1B dataset

2. NCEP/CPC L3 Half Hourly 4km Global (60S - 60N) Merged IR V1 (GPM-MERGIR): Produced by NOAA's NCEP in collaboration with NCAR, this dataset comprises globally merged infrared brightness temperature data from multiple geostationary satellites operated by the U.S., Europe, and Japan. It covers latitudes from 60°S to 60°N, with a spatial resolution of 4 km per pixel and half-hourly temporal grids (each file contains two grids). Preprocessing includes:
 - Correction for zenith angle dependence (mitigating geometric and radiometric errors).
 - Parallax correction (adjusting cloud top geolocation).

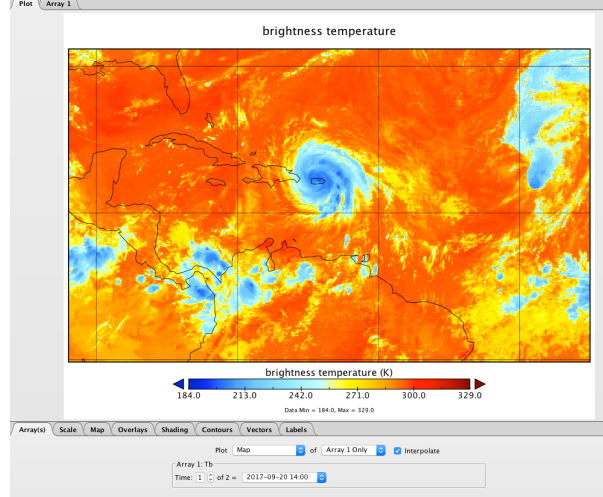


Figure 2.2: Representation of the NCEP/CPC level 3 dataset

Both datasets have undergone minimal processing, ensuring suitable spatial and temporal resolutions. The datasets however are quite different in format: the GPM Level 1B data is delivered in moving swaths, where sensor positions and wavelengths define the spatial distribution, while the NCEP/CPC dataset is gridded—each grid cell is a node with naturally correlated neighbors. Moreover, the GPM data is acquired by moving satellites, whereas the NCEP/CPC data is collected by geostationary satellites, simplifying graph-based modeling.

The datasets however are quite different in format: the first one is provided in moving swaths, while the second one is gridded. So, the concept of nodes of the network of measurements is quite different: in the first case we could consider the sensors along with the different wavelengths as the "spatial" distribution, and the concept of neighborhood is related to the fact that the sensors capture different (but correlated) measurements, with different wavelengths. On the other hand, the second dataset would be easier to model: it would be enough to model a single grid cell as a node, and then we know for sure that neighboring cells/nodes share similar values (since we are dealing with a continuous measurement) and they will mostly follow regular patterns over time. Moreover, while the first dataset is acquired by moving satellites, the second one is collected by geostationary satellites, which makes it even easier to be modeled using a graph-like structure.

2.3.1 NCEP/CPC dataset exploration

The dataset has a very detailed spatial and temporal resolution, so I had to subsample it. Namely, I considered the measurements taken in the first 4 months of the year 2020 and considered the central-southern part of Europe.

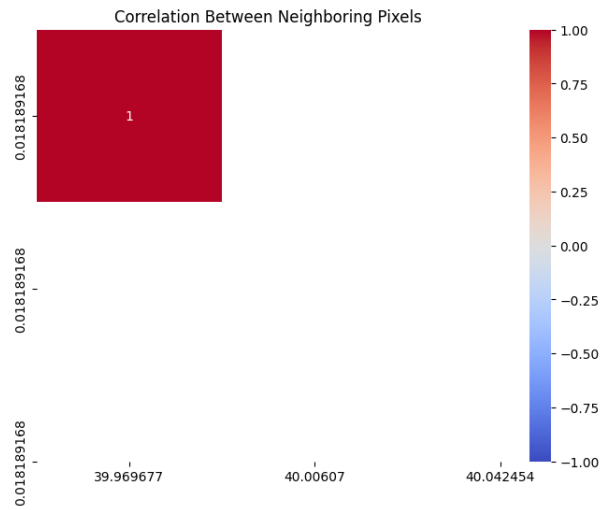


Figure 2.3: Neighboring pixels spatial correlation. It can be observed that there is indeed a high correlation

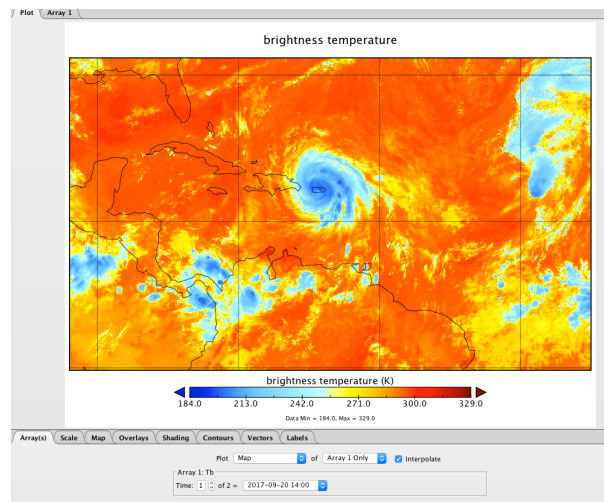


Figure 2.4: Frequency distribution of temperatures across a subset of the dataset. It can be observed that they are not position-dependent. As stated in the dataset documentation, they are mainly due to sensor observations not considered accurate enough

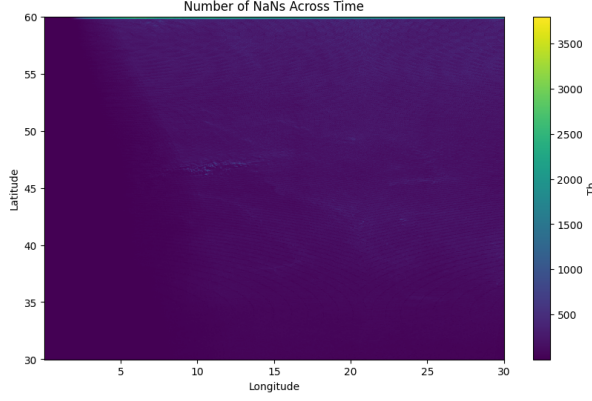


Figure 2.5: Null values distribution across the dataset

2.3.2 Potential attacks

The considered dataset plays a critical role in weather forecasting and climate monitoring by providing accurate infrared brightness temperatures over a wide geographical area. If this dataset were tampered with, several adverse outcomes could arise:

- **Misinterpretation of Cloud and Weather Patterns:** Injecting a bias into the brightness temperature data could lead to a misclassification of cloud cover and cloud-top temperatures. This, in turn, might cause weather models to misestimate the intensity and trajectory of storm systems. For example, even a small perturbation may result in forecast errors of up to 15–20% in extreme weather conditions [12].
- **Delayed or Inaccurate Weather Warnings:** Altered data can compromise the reliability of automated alert systems. Inaccurate or delayed warnings can have significant economic and human safety implications, particularly in regions prone to severe weather events [7].
- **Long-term Climate Model Degradation:** Satellite data is integral to the calibration of climate models. Systematic tampering, may skew long-term trends, thereby impacting climate projections and subsequent policy decisions.

An attacker who is able to perform Man-In-The-Middle attacks or to gain access to one or more satellites of this mission could manipulate data at their advantage. In particular, here is a list of attacks that they could perform at this stage:

- **Bias addition:** An attacker may add a small bias to the sensor measurements. This bias can even increase over time, simulating a gradual (or rapid) rise in the observed IR temperatures that might be nearly imperceptible.

$$\tilde{x}(t) = x(t) + \alpha t$$

- **Adversarial perturbation:** They could add some random noise to the observed data before it is sent to the central observation station. This data could also pass undetected to observers, but could seriously degrade the performance of machine learning models that are trained on this dataset, for example, to perform predictions

$$\tilde{x} = x + \epsilon \operatorname{sign}(\nabla_x \mathcal{L}(x, y))$$

- **Sensor spoofing:** Instead of modifying existing values, the attacker could also inject completely fake observation data at certain spatial locations.

$$\tilde{x}_i = \begin{cases} x_i^f, & \text{if } i \in S, \\ x_i, & \text{otherwise.} \end{cases}$$

- **Signal jamming:** The dataset comprises null values. However, they are an intrinsic feature of the dataset, since they are essentially values of measurements considered unreliable, mainly due to

sensor limitations. However, they present a certain distribution across the data, and if an attacker filled up the dataset of more invalid values, not only the resulting maps would become less detailed, but this could also make models trained on that dataset less performant, since those null values will somehow have to be interpolated, and will not hence be realistical data.

$$\tilde{x}_i = \begin{cases} \text{NaN}, & \text{if } i \in S, \\ x_i, & \text{otherwise.} \end{cases}$$

- **Temporal shifting:** consists in a manipulation of a time series by reordering its elements, thereby disrupting the temporal dependencies that models rely on. Specifically, given a time series of length TT , the attacker shifts the last nn elements to the beginning, effectively moving data points from the end of the series to the start. This backward shift can mislead models into learning incorrect temporal patterns, adversely affecting their performance.

$$\{x_i\}_{i=1}^T \rightarrow \{x_{T-n+i}\}_{i=1}^n \cup \{x_i\}_{i=1}^{T-n}$$

- **Spatial shifting:** this type of attack consists in shifting the measurements to arbitrary values

$$\tilde{x}(\mathbf{p}) = x(\mathbf{p} + \Delta\mathbf{p})$$

- **Miscalibration error injection:** The attacker can introduce a multiplicative error to simulate miscalibration of the sensor, scaling the measurement by a factor.

$$\tilde{x} = \alpha x$$

- **Replay attack:** this attack consists of repeating past observations in the present, and it could be performed in order to hide current events in a stealthier way

$$\tilde{x}(t) = x(t_0) \quad \text{for } t \in [t_1, t_2]$$

- **Backdoor attack:** This attack generally requires the attacker to have access to the training pipeline. In particular, it consists of adding some watermarks in the data, such that a network that will predict future values, in presence of such patterns the network will have a specific behavior desired by the attacker. However, if the attacker has insights on the type of model that is being trained on that dataset and some additional data, such as the temporal window size, they could craft poisoned data to make the model behave like they want in presence of such data.

$$\tilde{x} = x + \mathbf{1}_S \cdot z$$

Chapter 3

The Anomaly Detector

3.1 Proposed Solution: Unsupervised Training of STGAE

3.1.1 Graph Neural Networks

Graph Neural Networks (GNNs) are deep learning architectures tailored for graph-structured data, making them well-suited for tasks where relationships and dependencies are key — such as in social networks, recommendation systems, and knowledge graphs. To capture the connections and features of different nodes, GNNs examine the local neighborhood around each node.

Specifically, the neighborhood $N(i)$ of a node i is defined as the set of all nodes j that are directly connected to i by an edge (i.e. $N(i) = \{j \mid e_{i,j} \in E\}$.)

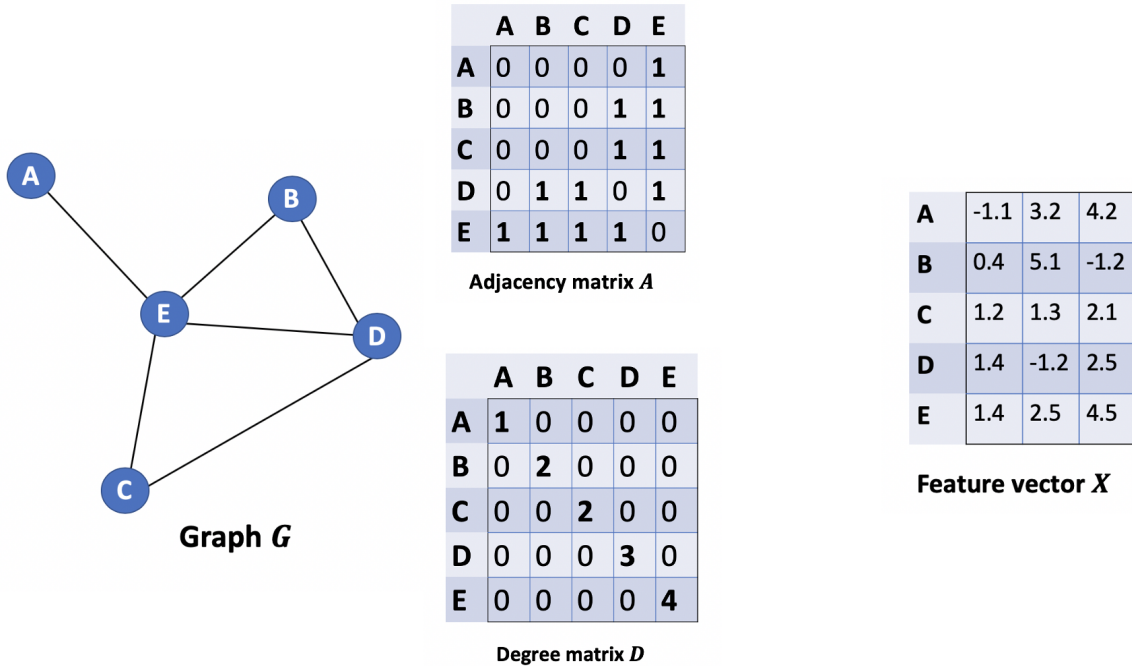


Figure 3.1: Representation of a GNN. The Adjacency matrix is a sparse matrix that contains cells with a value of 1 for nodes directly connected to each other

GNNs learn continuous feature representations for nodes by aggregating information from their neighbors. This process typically follows three main steps:

1. **Message Passing:** For every node, features from neighboring nodes are collected, transformed, and then transmitted back to the source node. This operation is executed in parallel for all nodes, ensuring that every neighborhood is processed.

2. **Aggregation:** An aggregation function (denoted as AGGREGATE) is applied to the set of messages from the neighboring nodes, summarizing the information.
3. **Update:** The node's current feature vector is combined—via operations such as addition or concatenation—with the aggregated messages to form an updated representation.

This process can be formally expressed as:

$$h_i^{(k+1)} = \text{UPDATE}^{(k)}\left(h_i^{(k)}, \text{AGGREGATE}^{(k)}(\{h_j^{(k)} : j \in N(i)\})\right),$$

where:

- k denotes the layer index, a hyperparameter chosen based on the graph and the specific task.
- The AGGREGATE function can be implemented in various ways (e.g., as the Mean, Max, a neural network, or even a recurrent neural network).
- UPDATE is the update function, e.g., Mean, Max, normalized sum, or NN.

Tasks may be categorized based on the level of prediction within the graph structure:

- Node-level tasks are concerned with predicting the identity or role of each node
- For an edge-level task, we want to predict the property or presence of edges in a graph.
- In a graph-level task, we predict a single property for a whole graph

Since our goal is to understand if we have anomalous nodes and which nodes are involved, node-level task is the most appropriate for this type of classification. Among all the types of GNNs, as already mentioned, I used STGAEs for detecting anomalies in this dataset.

3.1.2 Graph Auto Encoders

Graph autoencoders (GAEs) map graph data into a low-dimensional space and consists of two parts:

- Encoder E : transforms the graph input data x into a compressed or encoded representation $Z = E(X)$ (namely embedding), typically of lower dimensionality than the input. It captures the most important features of the input data.
- Decoder D : takes the encoded representation z and attempts to reconstruct the original graph input data from it. The goal is to produce an output graph $X' = D(E(X))$ that closely resembles the input graph $X(X \approx X')$.

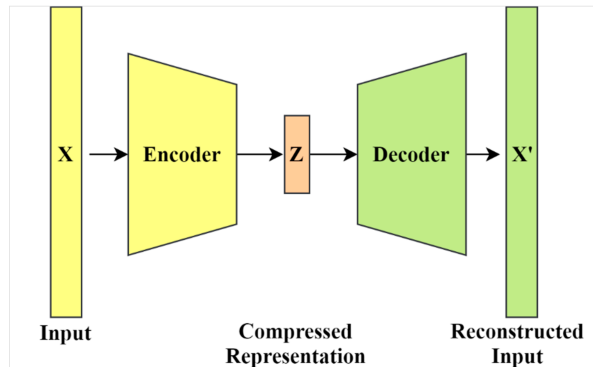


Figure 3.2: Representation of an auto-encoder

An auto-encoder is used for unsupervised learning, which means it does not need any label or target to learn from the data. It may learn efficient codings of the System Model data, which may be useful for dimensionality reduction, data compression, data generation, feature extraction, and more. Its primary objective is to minimize the reconstruction error, which indicates how well the decoder may recreate

the input from the encoded representation. This means that it learns to capture essential features and patterns in the data during the encoding process.

The encoder is usually implemented using Graph Convolutional Network layers (GCNConv), which takes each node and performs a weighted aggregation of features from its neighbors, combining them with the node's own features. The output is a lower-dimensional representation of the nodes, capturing both the feature and structural information of the input graph.

3.1.3 Spatio-Temporal Graph Auto Encoders

Spatio-Temporal Graph Auto Encoders (STGAEs) are designed to learn latent representations from graph-structured data that evolves over time: this allows them to analyze both spatial and temporal dimensions.

A STGAE is generally composed of three main components:

1. **Spatial Encoder:** it captures the inherent structure of the graph at a given timestamp. It commonly uses graph neural network layers such as Graph Convolutional Networks (GCNs) or Graph Attention Networks (GATs) to aggregate information from a node's local neighborhood.
2. **Temporal Encoder:** it processes sequences of graph snapshots (or temporal features associated with each node/edge). They are usually implemented with Recurrent Neural Networks (RNNs), Long Short-Term Memory networks (LSTMs), or Temporal Convolutional Networks (TCNs). This encoder learns how the relationships between nodes evolve and how these changes correlate with the system's behavior.
3. **Decoder:** It attempts to reconstruct the original input graph from the compressed spatio-temporal latent representation. By minimizing the reconstruction error during training, the model learns a latent space that faithfully represents the normal behavior of the system. During inference, deviations from this learned norm —reflected as a high reconstruction error- can be interpreted as anomalies.

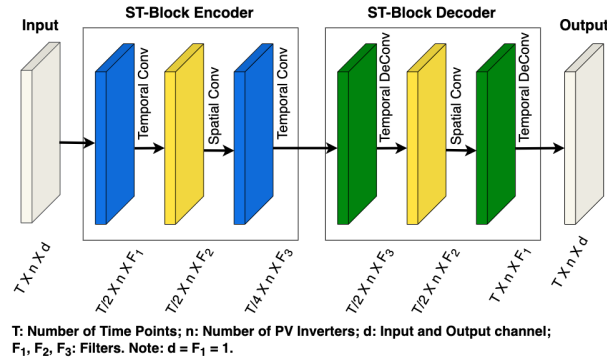


Figure 3.3: Representation of a STGAE: example of a Structure of the Spatial Layers and Temporal Layers

STGAEs extend the concept of auto encoders (neural networks that compress data into a lower-dimensional latent space and then reconstruct the original input) by operating directly on graph-structured data.

As already mentioned before, many systems (e.g., networks of IoT devices or communication networks) can naturally be modeled as graphs, where nodes represent entities (devices, sensors, etc.) and edges represent interactions or data flows. These interactions often change over time, and STGAES are specifically engineered to capture these dynamic patterns.

3.1.4 The proposed solution

The STGAE I built was composed by:

1. The Encoder

- 3 convolutional layers
- 1 transformer layer
- 1 convolutional layer

2. The Decoder

- 1 convolutional layer
- 1 transformer layer
- 3 convolutional layers

While setting up this network I had to be careful on the following aspects:

1. The dataset comprises null data. As already mentioned before, null data are not to be considered data to be discarded, but rather they represent sensor-specific features, and thus should not be eliminated or ignored. I decided then to model each graph node as a tuple composed by the temperature value and a mask for presence of null value
2. The values had to be normalized, not only to prevent features with larger ranges from dominating the optimization process [5], but also because with this method deviations (such as those introduced by poisoning attacks) become more pronounced relative to the expected range of values [2]
3. Due to the graph's large size and high memory demands, I was unable to construct extensive node neighborhoods and, as a result, had to increase the number of convolutional layers to capture relationships between farther nodes.
4. Limited available memory and computational constraints led me to adopt a TopKPooling layer, which also helped reduce redundancy in node connections.
5. The loss function had to account for both the temperature values and the latent mask representation to ensure accurate reconstruction and anomaly detection. So it was calculated as follows:

$$\mathcal{L} = w_T \cdot \mathcal{L}_T + w_M \cdot \mathcal{L}_M$$

where:

$$\mathcal{L}_T = \frac{1}{N} \sum_{i=1}^N (\hat{T}_i - T_i)^2 \text{ and } \mathcal{L}_M = \frac{1}{N} \sum_{i=1}^N (\hat{M}_i - M_i)^2$$

and

- $\hat{T}_i$ and T_i are the reconstructed and target temperature values.
- $\hat{M}_i$ and M_i are the reconstructed and target mask values.
- w_T and w_M are weighting factors.
- N is the total number of samples.

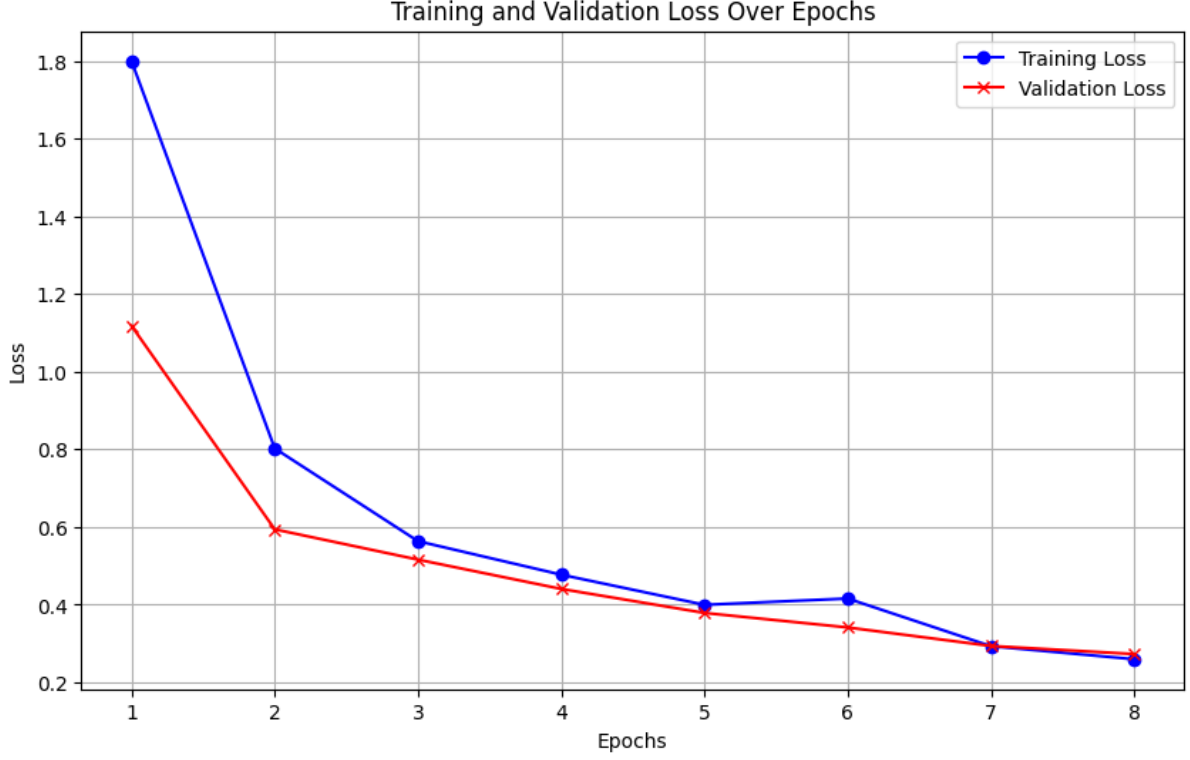


Figure 3.4: Training and validation loss for the STGAE

The model was trained for 8 epochs on the equivalent of 4 months of collected data. During the last epoch the network was trained on a larger sample of the data but with a lower learning rate.

3.1.5 Evaluation Metrics

The network's capability at detecting anomalous data was tested on a synthetic dataset that I generated, representing the anomalies that have been discussed in the previous chapter with various parameters. Considering the threshold of the reconstruction error to be 0.27 (which is the mean of the reconstruction error that non-anomalous data are characterized by), the overall accuracy was 77%

3.1.6 Results and Analysis

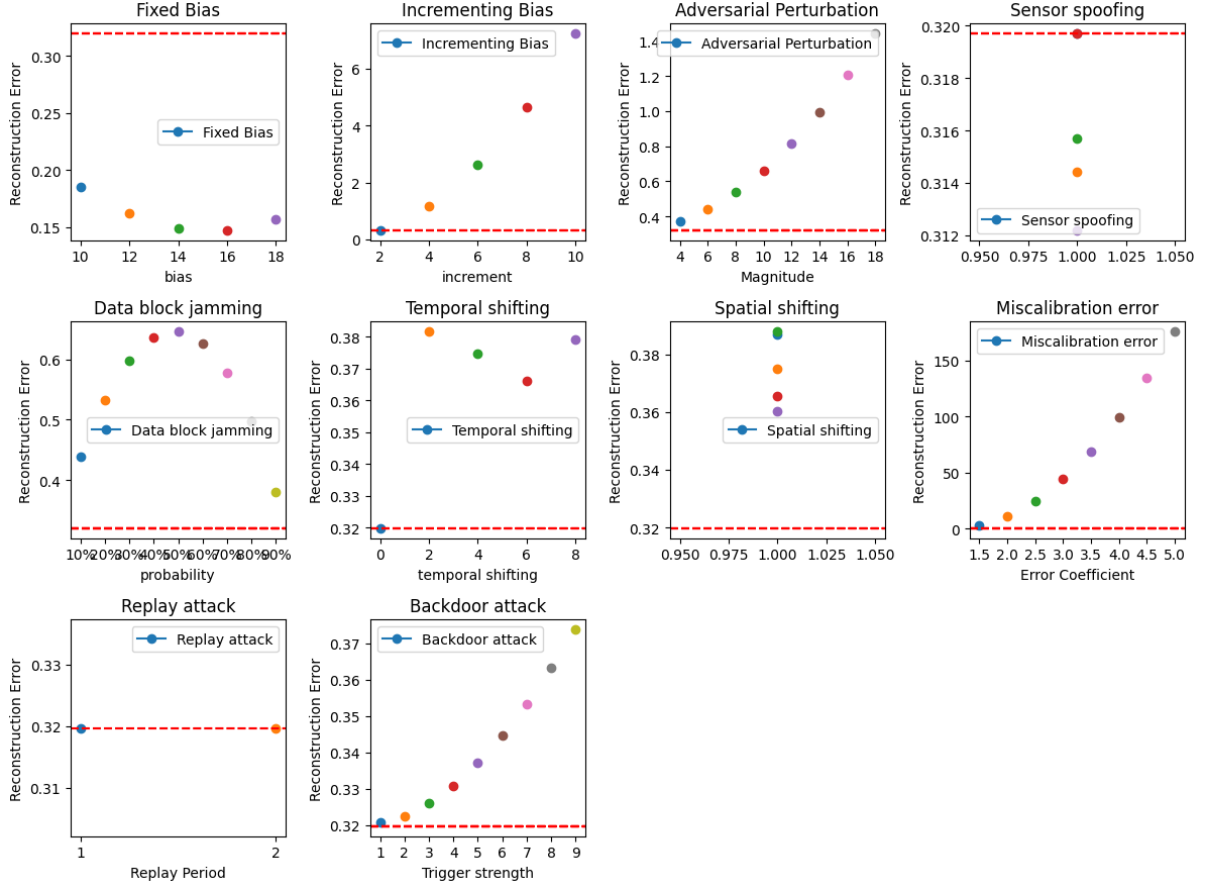


Figure 3.5: Performance of the STGAE against different data poisoning attacks with various parameters

The accuracy alone does not give much insight on the model’s capabilities. In fact, it performed very well on backdoor attacks (it had a mean of 1.6 of reconstruction error), on miscalibration simulation (with a mean of 160), performed worse in case sensor spoofing (with 0.21). However, the network performed better when the bias increased over time. The main problem with the considered network lies probably in the lack of parameters that a model like STGAEs lacks the capacity to correlate farther nodes (as should do in this particular case).

Chapter 4

Conclusion

4.1 Summary of Findings

The proposed framework effectively identified the majority of the specified attack patterns with acceptable precision. However, it exhibited limitations in capturing correlations over extended regions of the map, rendering it less effective against poisoning attacks that target large contiguous areas, such as sensor spoofing.

4.2 Improvements and Future Developments

To enhance the framework's performance, future work could incorporate features like sparsity, which may facilitate more interconnected neighborhoods. Additionally, addressing automatic poisoning attacks could be achieved by employing Generative Adversarial Networks (GANs) to generate synthetic anomalous data from existing anomalous datasets; in fact, recent studies have explored the use of GANs to generate synthetic anomalous data, which can be utilized to enhance the robustness of detection systems. For instance, research has investigated data poisoning attacks within federated learning systems, where GANs are employed to produce synthetic malicious data, thereby challenging the integrity of the learning process.[13] Additionally, the application of GANs in generating synthetic anomalous data has been explored to improve the detection of data poisoning attacks. By training GANs on existing anomalous data, it is possible to create new instances that can be used to test and strengthen detection frameworks. [14]. Finally, applying the approach of this project to measure correlations between satellite swaths, particularly among geostationary satellites that share overlapping regions, could provide valuable insights.

Bibliography

- [1] Ibrahim Alrashdi, Mahmoud Alshawabkeh, Magdy Bayoumi, Nasir Ghani, Thomas Shrimpton, and Roy Campbell. Preventing data tampering in iot networks. *IEEE Access*, 7:155666–155681, 2019.
- [2] V. Chandola, A. Banerjee, and V. Kumar. Anomaly detection: A survey. *ACM Computing Surveys (CSUR)*, 41(3):1–58, 2009.
- [3] Mohamed Amine Ferrag, Othmane Friha, Djallel Hamouda, Leandros Maglaras, and Helge Janicke. Edge-IIoTset: A New Comprehensive Realistic Cyber Security Dataset of IoT and IIoT Applications: Centralized and Federated Learning, 2022.
- [4] Mengnan Gao, Lifa Wu, Qi Li, and Wei Chen. Anomaly traffic detection in iot security using graph neural networks. *Journal of Information Security and Applications*, 72:103532, 2023.
- [5] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, Cambridge, MA, 2016.
- [6] Kyle Hundman, Valentino Constantinou, Christopher Laporte, Ian Colwell, and Tom Soderstrom. Detecting spacecraft anomalies using lstms and nonparametric dynamic thresholding. *arXiv preprint arXiv:1802.04431*, 2018.
- [7] M. Jones and S. Patel. Cybersecurity challenges in satellite operations. *IEEE Transactions on Aerospace and Electronic Systems*, 55:567–578, 2019.
- [8] Y. Liu, Y. Zhang, J. Wang, X. Li, and H. Zhang. Anomaly detection in iot networks: A graph-based approach. In *2022 IEEE International Conference on Communications (ICC)*, pages 1–6, 2022.
- [9] Yang Liu, Ming Li, Xiaoli Zhang, and Jianyu Wang. Satellite on-orbit anomaly detection method based on a dynamic gated recurrent unit model. *IEEE Access*, 9:57472–57482, 2021.
- [10] Yuhang Liu, Xiang Zhang, Jian Wang, Kezhi Li, Nan Zhang, Yichen Zhang, and Xuemin (Sherman) Zhang. On the interplay of artificial intelligence and space-air-ground integrated networks: A survey. *IEEE Communications Surveys & Tutorials*, 26(1):1–36, 2024.
- [11] Joseph Rose, Matthew Swann, Gueltoum Bendiab, Stavros Shiaeles, and Nicholas Kolokotronis. Intrusion detection using network traffic profiling and machine learning for iot. *arXiv preprint arXiv:2109.02544*, 2021.
- [12] J. Smith and A. Doe. Data integrity in satellite observations. *Journal of Remote Sensing*, 45:123–134, 2018.
- [13] L. Wang, H. Zhang, S. Li, and Y. Liu. Gan-driven data poisoning attacks and their mitigation in federated learning systems. *Electronics*, 12(8):1805, 2023.
- [14] Y. Zhang, X. Wang, J. Li, and Y. Zhang. A gan-based data poisoning attack against federated learning systems. *arXiv preprint arXiv:2405.11440*, 2024.